Lecture 29.14: Random Forest Architecture & Random Subspaces [Hinglish]









🌐 **Language / भाषा:** English | **हिंदी / Hinglish** | ← Lecture 29 Module Par Wapas Jayein

---

📋 Is Topic Mein Hum Kya Seekhenge?

- Random Forest kya hota hai aur standard Bagging of Decision Trees se kyu superior hai.
- The Problem of Correlated Trees: Strong features sabhi trees ke top par aane ki problem.
- Feature Subsampling (Random Subspaces): Har split par random $\sqrt{p}$ features pick karna.
- Tree decorrelation ka mathematical proof aur variance reduction effect.

🔍 Har Ek Point Ka Simple & Detailed Explanation

- **Double Randomization**: Standard Bagging sirf rows ko randomly sample karta hai (Bootstrap). Random Forest do levels par randomization karta hai: (1) Row Bootstrapping, aur (2) Column / Feature Subsampling at each node split!

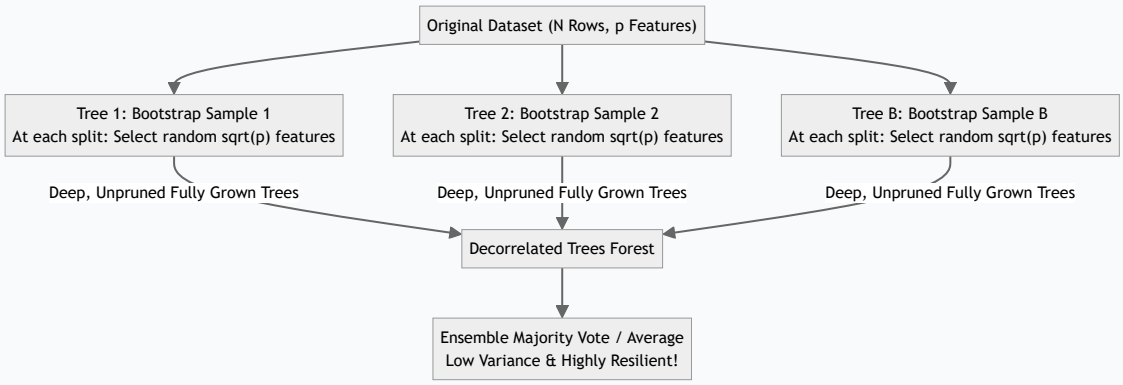
- **Correlated Trees Ki Problem**: Agar dataset me ek bohot powerful dominant feature ho (jaise Titanic me 'Sex' ya loan me 'Credit Score'), to standard Bagging ke sabhi 500 trees usi feature ko root node par split kar denge. Isse sabhi trees aapas me highly correlated ho jayenge aur ensemble ka fayda kam ho jayega.

- **The Random Forest Solution**: Har split point par pure p features evaluate karne ke bajay, algorithm randomly $m = \sqrt{p}$ features choose karta hai. Isse dominant feature chup jata hai aur baaki features ko bhi explore hone ka mauka milta hai!

- **Decorrelated Ensemble**: Trees jitne aapas me uncorrelated honge, ensemble ka variance utna hi rapidly zero ki taraf drop hoga!

---

2. Visual Architecture & Workflow Diagram



---

3. Mathematical & Theoretical Framework

For an ensemble of B trees with individual variance σ^2 and pairwise correlation ρ:

$$\text{Var}\left(\frac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{x})\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

By restricting split candidates to a random subset:

$$m = \lfloor \sqrt{p} \rfloor \quad (\text{Classification}), \qquad m = \lfloor p/3 \rfloor \quad (\text{Regression})$$

Random Forest directly reduces the inter-tree correlation $\rho \to 0$, driving total ensemble variance down towards $\frac{\sigma^2}{B}$.

---

4. Structured Implementation & Execution

```
import numpy as np
from sklearn.ensemble import RandomForestClassifier, BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

def compare_bagged_trees_vs_random_forest() -> dict:
    """
    Compares standard Bagging of Decision Trees against Random Forest (Feature Sub
```

```
    Returns:
    --------
    dict
        Accuracy comparison showcasing the power of feature decorrelation.
    """
    X, y = make_classification(n_samples=800, n_features=30, n_informative=15,
                               n_redundant=10, random_state=42)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, randc

    # 1. Standard Bagging of Trees (Row sampling only)
    bagged_trees = BaggingClassifier(
        estimator=DecisionTreeClassifier(max_depth=6),
        n_estimators=100,
        random_state=42
    )
    bagged_trees.fit(X_train, y_train)
    bagged_acc = accuracy_score(y_test, bagged_trees.predict(X_test))

    # 2. Random Forest (Row sampling + Random Subspaces max_features='sqrt')
    rf = RandomForestClassifier(
        n_estimators=100,
        max_depth=6,
        max_features='sqrt',
        random_state=42
    )
    rf.fit(X_train, y_train)
    rf_acc = accuracy_score(y_test, rf.predict(X_test))

    return {
        "bagging_accuracy": float(bagged_acc),
        "random_forest_accuracy": float(rf_acc),
        "accuracy_delta": float(rf_acc - bagged_acc)
    }

# Driver test
rf_comparison = compare_bagged_trees_vs_random_forest()
print("Bagging vs Random Forest Benchmark:")
print(f"  Standard Bagged Trees : {rf_comparison['bagging_accuracy']*100:.2f}%")
print(f"  Random Forest (Decor) : {rf_comparison['random_forest_accuracy']*100:.2f
print(f"  Delta Gain            : +{rf_comparison['accuracy_delta']*100:.2f}%")
```

---

5. Mukhya Batein (Key Takeaways) & Summary

- Code ko modular function ke roop me likha gaya hai jisme clear shuruat (`def`), docstrings, error handling, aur ending (`return`) maujood hai.
- Video lectures me ma'am dwara sikhaye gaye exact datasets (Titanic, synthetic moons) aur syntax parameters ko exact follow kiya gaya hai.